Autor: Juan Gabriel Galarza Claros

Desarrollo de Interfaces – 2º DAM
UT2 – Bloque 2 – Práctica 1

Memoria técnica: componente visual autónomo en JavaFX

Proyecto: app del restaurante El Buen Gusto (paquete com.elbuengusto.componentes)

1. El componente y lo que hace

Para esta práctica he hecho el componente **LoginComponente**, que es la pantalla de login de la app de El Buen Gusto (la que diseñé antes en los wireframes y el mockup) pero metida en un componente aparte. La idea es que se pueda usar en la ventana principal como si fuera un control más de JavaFX, sin tener todo el código del login mezclado en el HelloController.

Cómo lo he montado

El FXML del componente es login-componente.fxml y en vez de poner un StackPane normal como raíz he usado <fx:root type="javafx.scene.layout.StackPane">. He elegido StackPane porque así puedo poner detrás una capa con los iconos de cubiertos de decoración y encima el formulario. Luego la clase LoginComponente extiende de StackPane, y en el constructor cargo el FXML con fxmlLoader.setRoot(this) y fxmlLoader.setController(this). Con eso la propia clase hace de raíz y de controlador a la vez, y la vista y la lógica quedan juntas.

También tiene su propia hoja de estilos, login-estilo.css, con los colores que usé en el mockup: el fondo crema (#F6F5EC) y el verde oliva de los botones (#556B2F), que cambia a #647D38 al pasar el ratón por encima.

Validación del botón

En el initialize() he hecho un binding para que el botón de confirmar esté desactivado mientras falte el correo o la contraseña:

```
btnConfirmar.disableProperty().bind(txtCorreo.textProperty().isEmpty()
    .or(txtPassword.textProperty().isEmpty()));
```

Así no hace falta comprobar nada a mano. En cuanto se escribe algo en los dos campos el botón se activa solo, y si se borra uno se vuelve a desactivar. El botón de crear cuenta y el enlace de recuperar contraseña también se gestionan dentro del propio componente (de momento solo sacan un mensaje por consola).

Propiedades personalizables

Para poder cambiar los textos desde fuera he puesto dos propiedades siguiendo el estándar JavaBean, es decir, con sus tres métodos (xxxProperty(), getXxx() y setXxx()):

- titulo, que va enlazada al Label de arriba.
- textoBoton, que va enlazada al texto del botón.

De esta forma se pueden poner como atributos en hello-view.fxml o cambiar desde Scene Builder. En la tabla pongo todo lo que el componente deja usar desde fuera:

Qué es	Métodos	Para qué lo uso
Propiedades JavaBean	tituloProperty(), getTitulo(), setTitulo() textoBotonProperty(), getTextoBoton(), setTextoBoton()	Cambiar el título y el texto del botón desde el FXML o desde código.
Evento del botón	onConfirmarActionProperty(), getOnConfirmarAction(), setOnConfirmarAction()	Que HelloController se entere de que se ha pulsado confirmar y saque el mensaje por consola.
Consultar el usuario	getUsuarioIngresado()	Coger el nombre del usuario a partir del correo que ha escrito (como String).

2. Comunicación con el exterior sin romper la encapsulación

Lo más fácil habría sido poner los controles como públicos o hacer métodos tipo getBtnConfirmar() o getTxtCorreo() para usarlos desde el HelloController, pero eso es justo lo que no hay que hacer. Si la ventana principal puede tocar los controles de dentro, podría cambiarles el estilo, quitarlos, cargarse el binding del botón, etc. Y además el HelloController dependería de cómo está hecho el FXML por dentro, así que cualquier cambio en el componente podría romperlo.

Por eso he intentado que el componente funcione como una caja negra. Para conseguirlo he hecho lo siguiente:

Controles privados

Todos los controles que vienen del FXML (lblTitulo, txtCorreo, txtPassword, btnCrearCuenta, linkRecuperar y btnConfirmar) son private y no tienen ningún getter público, así que desde otra clase no se puede llegar a ellos.

Evento propio para el botón

Para avisar de que se ha pulsado confirmar sin sacar el botón fuera, he creado la propiedad onConfirmarAction, que es un ObjectProperty<EventHandler<ActionEvent>>. Cuando se pulsa el botón se ejecuta el método privado manejarConfirmacion(ActionEvent), y este mira si alguien ha registrado un handler

con setOnConfirmarAction(...). Si lo hay, llama a su handle(event). El HelloController registra ahí lo que quiere hacer (sacar el mensaje por consola) y no necesita saber qué botón ha sido.

Devolver datos y no controles

Cuando la ventana principal necesita saber quién ha iniciado sesión, llama a getUsuarioIngresado(). Este método coge lo que hay escrito en el campo del correo, se queda con la parte de antes de la @ y le pone la primera letra en mayúscula (por ejemplo, de raul@gmail.com sale Raul). Lo devuelve como String, no el TextField. El HelloController usa ese nombre para sacarlo por consola y para ponerlo en el perfil de arriba a la derecha, junto al avatar y el ¡Bienvenido!. Y para cambiar los textos del componente se usan las propiedades de antes, nunca los controles.

Conclusión

Con esto, si más adelante quisiera cambiar algo por dentro del componente, por ejemplo usar otro control en vez del botón de confirmar o que el login se confirme también al dar a Enter en la contraseña, no tendría que tocar nada ni en HelloController ni en hello-view.fxml. Lo único que conoce la ventana principal son los métodos públicos del componente, y mientras esos no cambien todo sigue funcionando.